(Day2)Tshark进程间通信

#C #Cpp

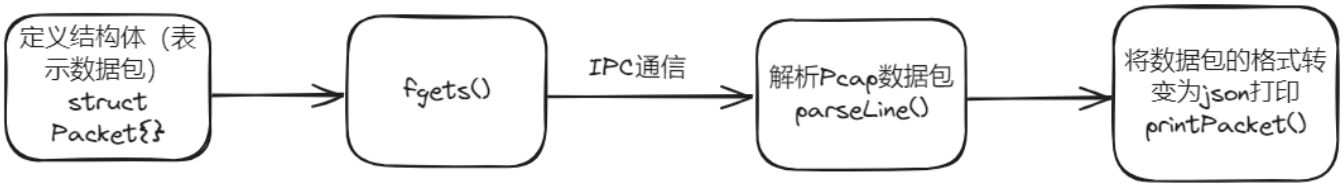
定义

进程间通信(Inter-Process Communication),又称为IPC，是值通过不同进程间交换数据的机制。在操作系统中，常见的方式包括：

- 管道（Pipes) : 单向数据流通道，父子进程可以使用；
- 命名管道(FIFO): 支持不同进程间的通信的管道；
- 共享内存 (SharedMemory) : 多个进程共享同一段内存，速度快但需要同步控制。
- 消息队列（MessageQueues) : 用于发送和接收数据块。
- 套接字（Sockets) : 用于本地或网络通信
  本次开发选择 `管道（Pipe）` 。

阶段开发原理

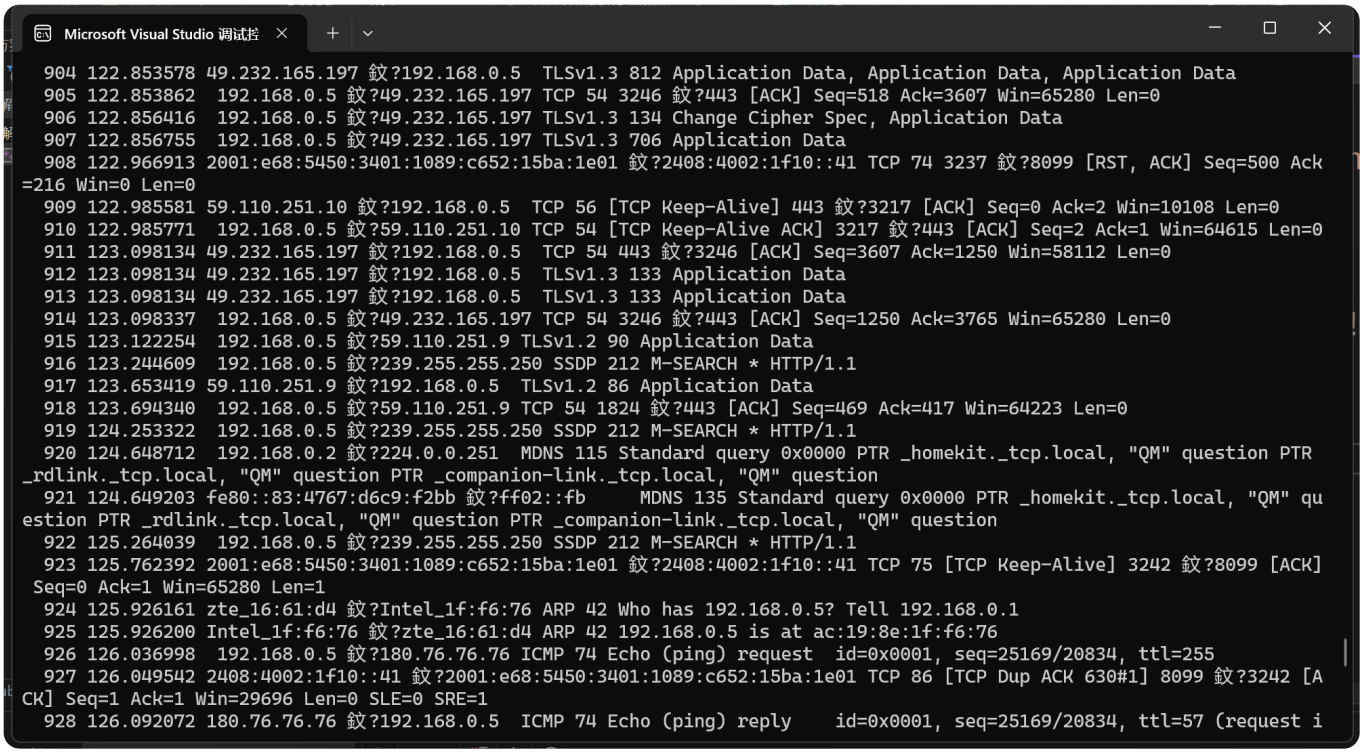
大概流程



所以如果更改了数据包的解析的内容，需要修改 `parseLine()` 和 `printPacket()` 这两个函数内部的代码逻辑。

乱码问题

编写了个demo，读取文件，但是显示乱码：



代码在读取 `tshark.exe` 输出时出现乱码，主要原因可能是 **编码不一致**（例如 `tshark` 输出的文本是 UTF-8 编码，而 Windows 控制台默认使用本地代码页如 GBK）。以下是解决方案：

- 解决方式一

```
#include <windows.h>  // 需要包含此头文件

int main() {
    // 设置控制台输出为 UTF-8 编码
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);

    const char* command = "D://programs//wireshark//tshark.exe -r D://Mylearn//EasyT
    // ... 其余代码不变 ...
}
```

- 解决方式二
  检查这个tshark的实际输出编码

```
const char* command = "D://programs//wireshark//tshark.exe -r D://Mylearn//EasyTshar
```

部分代码实现过程

需要导入的库

```
#include <iostream>
#include <cstdio>
#include <Windows.h>
#include <vector>
#include <sstream>

#include "...\EasyTshark\libs\rapidjson\document.h"
#include "...\Mylearn\EasyTshark\libs\rapidjson\writer.h"
#include "...\Mylearn\EasyTshark\libs\rapidjson\prettywriter.h"
#include "...\Mylearn\EasyTshark\libs\rapidjson\stringbuffer.h"
```

定义的结构体

```
// 定义一个结构体，用来表示一个数据包
struct Packet {
    int frame_number;   // 数据包编号
    std::string time;   //数据包的时间戳
    std::string src_ip; //源IP地址
    std::string src_port; //源IP端口
    std::string dst_ip; //目的IP地址
    std::string dst_port; // 目的IP地址
    std::string protocol; //协议
    std::string info; // 数据包的概要信息
};
```

fgets()函数

这段在 `main` 函数里：

```
 // 读取pcap文件，并输出
    char buffer[1024];
    while (fgets(buffer, sizeof(buffer), pipe) != nullptr) {
        Packet packet;
        parseLine(buffer, packet);
        packets.push_back(packet);
```

```
    }
```

解析函数

```
// 解析数据包
void parseLine(std::string line, Packet& packet) {

    if (line.back() == '\n') {
        line.pop_back();
    }
    std::stringstream ss(line);
    std::string field;
    std::vector<std::string> fields;

    while (std::getline(ss, field, '\t')) {  // 假设字段用 tab 分隔
        fields.push_back(field);
    }

    if (fields.size() >= 8) {
        packet.frame_number = std::stoi(fields[0]);
        packet.time = fields[1];
        packet.src_ip = fields[2];
        packet.src_port = fields[3];
        packet.dst_ip = fields[4];
        packet.dst_port = fields[5];
        packet.protocol = fields[6];
        packet.info = fields[7];
    }
    //}else{
    //    // 处理错误或日志记录
    //    std::cerr << "Warning: Expected at least 6 fields, but found" << fields.si
    //}
}
```

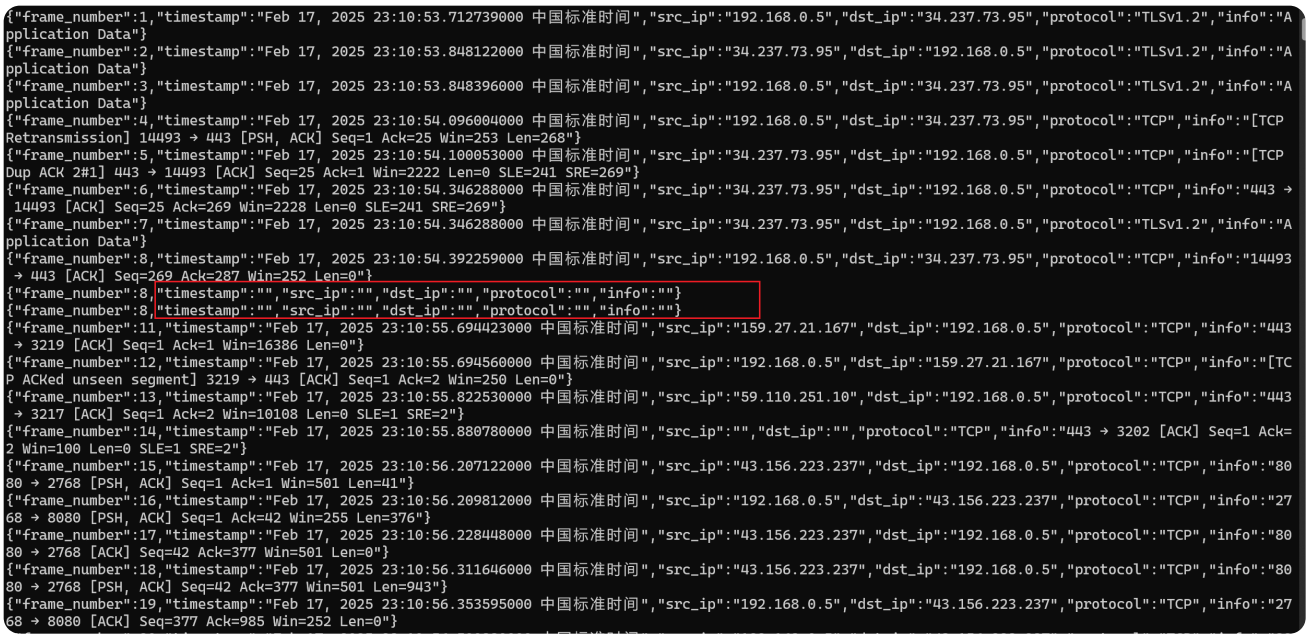
转变为json函数

```
// 将数据包的格式转变为json并打印
void printPacket(const Packet& packet) {
```

```
    // 构建JOSN对象
    rapidjson::Document pktObj;
    rapidjson::Document::AllocatorType& allocator = pktObj.GetAllocator();

    // 设置JSON为object对象类型
    pktObj.SetObject();

    //添加JSON字段
    pktObj.AddMember("frame_number", packet.frame_number, allocator);
    pktObj.AddMember("timestamp",rapidjson::Value(packet.time.c_str(),allocator),all
    pktObj.AddMember("src_ip",rapidjson::Value(packet.src_ip.c_str(),allocator),allo
    pktObj.AddMember("src_port", rapidjson::Value(packet.src_port.c_str(), allocator
    pktObj.AddMember("dst_ip", rapidjson::Value(packet.dst_ip.c_str(),allocator), al
    pktObj.AddMember("dst_port", rapidjson::Value(packet.dst_port.c_str(), allocator
    pktObj.AddMember("protocol", rapidjson::Value(packet.protocol.c_str(),allocator)
    pktObj.AddMember("info", rapidjson::Value(packet.info.c_str(),allocator), alloca

    // 序列化为JSON字符串
    rapidjson::StringBuffer buffer;
    rapidjson::Writer<rapidjson::StringBuffer>writer(buffer);
    pktObj.Accept(writer);

    // 打印JSON输出
    std::cout << buffer.GetString() << std::endl;


}
```

作业二

1. 不知道大家发现没有，上面输出的结果中，有些数据包的源IP和目的IP是空的，这是怎么回事呢？请自己研究一下，然后回答。下节课我会来解答这个问题。
   如下图，有的地址源IP和目的IP确实是空的。



有可能是因为：

- 非IP协议的数据包：不是所有的网络流量都是基于IP协议的。比如链路层(Layer2) 协议如ARP或者以太网帧本身并不含IP层信息。因此，当解析这些非IP协议的数据包时，自然不会有源IP或目的IP地址。
- IPV6 vs IPV4: 也可能是指定的IPV4字段，捕获的数据中包含了IPV6流量，对于IPV6数据包来说，`ip.src` 和 `ip.dst` 将不会被填充。

2. 上面输出的数据包信息，没有端口的信息，请增加端口信息，然后以JSON形式打印输出数据包编号、时间、源IP、源端口、目的IP、目的端口、协议、数据包概要信息这些字段。
如下：

```
{
    "frame_number":13,
    "timestamp":"Feb 17, 2025 23:10:55.822530000 中国标准时间",
    "src_ip":"59.110.251.10",
    "src_port":"443",
    "dst_ip":"192.168.0.5",
    "dst_port":"3217",
    "protocol":"TCP",
    "info":"443 → 3217 [ACK] Seq=1 Ack=2 Win=10108 Len=0 SLE=1 SRE=2"
}
```

最后整体的打印结果


```
{"frame_number":4,"timestamp":"Feb 17, 2025 23:10:54.096004000 中国标准时间","src_ip":"192.168.0.5","src_port":"14493","dst_ip":"34.237.73.95","dst_port":"443","protocol":"TCP","info":"[TCP Retransmission] 14493 → 443 [PSH, ACK] Seq=1 Ack=25 Win=253 Len=268"}
{"frame_number":5,"timestamp":"Feb 17, 2025 23:10:54.100053000 中国标准时间","src_ip":"34.237.73.95","src_port":"443","dst_ip":"192.168.0.5","dst_port":"14493","protocol":"TCP","info":"[TCP Dup ACK 2#1] 443 → 14493 [ACK] Seq=25 Ack=1 Win=2222 Len=0 SLE=241 SRE=269"}
{"frame_number":6,"timestamp":"Feb 17, 2025 23:10:54.346288000 中国标准时间","src_ip":"34.237.73.95","src_port":"443","dst_ip":"192.168.0.5","dst_port":"14493","protocol":"TCP","info":"443 → 14493 [ACK] Seq=25 Ack=269 Win=2228 Len=0 SLE=241 SRE=269"}
{"frame_number":7,"timestamp":"Feb 17, 2025 23:10:54.346288000 中国标准时间","src_ip":"34.237.73.95","src_port":"443","dst_ip":"192.168.0.5","dst_port":"14493","protocol":"TLSv1.2","info":"Application Data"}
{"frame_number":8,"timestamp":"Feb 17, 2025 23:10:54.392259000 中国标准时间","src_ip":"192.168.0.5","src_port":"14493","dst_ip":"34.237.73.95","dst_port":"443","protocol":"TCP","info":"14493 → 443 [ACK] Seq=269 Ack=287 Win=252 Len=0"}
{"frame_number":8,"timestamp":"","src_ip":"","src_port":"","dst_ip":"","dst_port":"","protocol":"","info":""}
{"frame_number":8,"timestamp":"","src_ip":"","src_port":"","dst_ip":"","dst_port":"","protocol":"","info":""}
{"frame_number":11,"timestamp":"Feb 17, 2025 23:10:55.694423000 中国标准时间","src_ip":"159.27.21.167","src_port":"443","dst_ip":"192.168.0.5","dst_port":"3219","protocol":"TCP","info":"443 → 3219 [ACK] Seq=1 Ack=1 Win=16386 Len=0"}
{"frame_number":12,"timestamp":"Feb 17, 2025 23:10:55.694560000 中国标准时间","src_ip":"192.168.0.5","src_port":"3219","dst_ip":"159.27.21.167","dst_port":"443","protocol":"TCP","info":"[TCP ACKed unseen segment] 3219 → 443 [ACK] Seq=1 Ack=2 Win=250 Len=0"}
{"frame_number":13,"timestamp":"Feb 17, 2025 23:10:55.822530000 中国标准时间","src_ip":"59.110.251.10","src_port":"443","dst_ip":"192.168.0.5","dst_port":"3217","protocol":"TCP","info":"443 → 3217 [ACK] Seq=1 Ack=2 Win=10108 Len=0 SLE=1 SRE=2"}
{"frame_number":14,"timestamp":"Feb 17, 2025 23:10:55.880780000 中国标准时间","src_ip":"","src_port":"443","dst_ip":"","dst_port":"3202","protocol":"TCP","info":"443 → 3202 [ACK] Seq=1 Ack=2 Win=100 Len=0 SLE=1 SRE=2"}
{"frame_number":15,"timestamp":"Feb 17, 2025 23:10:56.207122000 中国标准时间","src_ip":"43.156.223.237","src_port":"8080","dst_ip":"192.168.0.5","dst_port":"2768","protocol":"TCP","info":"8080 → 2768 [PSH, ACK] Seq=1 Ack=1 Win=501 Len=41"}
{"frame_number":16,"timestamp":"Feb 17, 2025 23:10:56.209812000 中国标准时间","src_ip":"192.168.0.5","src_port":"2768","dst_ip":"43.156.223.237","dst_port":"8080","protocol":"TCP","info":"2768 → 8080 [PSH, ACK] Seq=1 Ack=42 Win=255 Len=376"
```


小结

- 熟悉了进程间的通信；
- 梳理了从定义需要解析的数据包结构，然后到解析，最后打印的过程；
- 找出了源IP和目的IP存在空白的原因；
- 解决了一开始的解析数据包的乱码问题；